KEPCO E&C · 계측 직렬 필기

2026 하반기 · 필기 10.24 (토)

DAY 5

10.23 (금)

읽기 → 가리기 → 오른쪽에 쓰기 → 확인 → 지우고 다시

05 디지털 논리 · 컴퓨터 구조

가산기·감산기에서 operand와 캐시까지, 쓰면서 외우는 핵심

20	14	6	30	6h 10m
단원	A 필수	B 선택	공식 카드	첫 학습

색깔 읽는 법

노랑 형광	통암기 문장 · 단원 첫 정의. 이것만 먼저 외워도 뼈대가 잡힘
분홍 형광	"아니다·혼동·틀리" 등 헷갈림 주의 문장
공식 카드	가리고 오른쪽 패드에 기호·단위·조건까지 써보기
빨강 박스	함정. 시험 당일 아침에 이 박스만 훑기
초록 박스	짧은 예제. 풀이 흐름만 1회 확인
보라 박스	가리고 답하기. 정답은 권말 링크

DAY는 권장 과목 순서이며 전체 확장 내용을 하루에 마치는 기한이 아닙니다. 시간대와 분량은 참고값입니다. 원본의 세부 범위까지 읽을 때는 여러 날에 나누어 학습하세요.

목차 · 회독 체크

단원 이름을 누르면 해당 쪽으로 이동합니다. 오른쪽 동그라미는 회독할 때마다 펜으로 채우세요.

ID	단원	우선	시간대	쪽	1 2 3회독
D01	진법과 비트: 수의 크기부터 읽기	A	오전 · 20분	5	○ ○ ○
D02	음수 표현과 2의 보수	B	오전 · 15분	7	○ ○ ○
D03	그레이코드: 인접 상태는 한 비트만 바뀐다	A	오전 · 20분	9	○ ○ ○
D04	논리게이트와 진리표	A	오전 · 20분	12	○ ○ ○
D05	드모르간과 범용 게이트	A	오전 · 20분	17	○ ○ ○
D06	불 대수·최소항과 논리식 간소화	A	오전 · 20분	21	○ ○ ○
D07	카르노맵: 묶는 규칙과 남는 변수	A	오전 · 20분	26	○ ○ ○
D08	반가산기와 전가산기	A	오전 · 20분	29	○ ○ ○
D09	반감산기와 전감산기	A	오전 · 20분	32	○ ○ ○
D10	선택기·해독기·부호기 구분	B	오전 · 15분	35	○ ○ ○
D11	래치·플립플롭과 다음 상태	A	오후 · 20분	37	○ ○ ○
D12	카운터·시프트 레지스터	B	오후 · 15분	39	○ ○ ○
D13	CMOS 게이트를 스위치로 읽기	B	오후 · 15분	41	○ ○ ○
D14	폰 노이만 구조와 컴퓨터의 구성	A	오후 · 20분	42	○ ○ ○
D15	명령어: opcode와 operand	A	오후 · 20분	45	○ ○ ○
D16	레지스터와 명령 실행 순서	A	오후 · 20분	48	○ ○ ○
D17	기억장치 계층과 RAM·ROM	A	오후 · 20분	51	○ ○ ○
D18	캐시: 지역성·적중률·쓰기 정책	A	오후 · 20분	54	○ ○ ○
D19	인터럽트·DMA·입출력 방식	B	오후 · 15분	57	○ ○ ○

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

ID	단원	우선	시간대	쪽	1 2 3회독
D20	ADC·DAC·표본화의 최소 개념	B	오후 · 15분	60	◯ ◯ ◯
	공식 총정리 · 백지 테스트			62	
	함정 총정리 · 시험 당일 아침용			65	
	백지 회상 문항			68	
	정답 모음			71	

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

이 권에서 잡을 것

이 권은 추가 복기의 전감산기·카르노맵·그레이코드·operand·캐시와 상반기 복기의 반가산기·범용 게이트·폰 노이만 구조를 연결한다. 계산보다 구분과 짧은 진리표를 우선한다.

A는 5일 안에 우선 회상할 내용, B는 A를 한 번 익힌 뒤 볼 보완 내용이다. 본문을 읽은 뒤 오른쪽 필기 공간에 공식·진리표·핵심 문장을 가리고 재현한다. 문제의 정답은 권 뒤쪽에 모았다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

D01 진법과 비트: 수의 크기부터 읽기

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 2개

이진수의 각 자리는 오른쪽부터 1, 2, 4, 8, 16의 자릿값을 갖는다. 비트는 0 또는 1 한 자리이며, 바이트는 8비트다. 1011이라는 모양만 보고 십진수 천십일로 읽으면 안 된다. 진법의 아래첨자나 문제의 표현을 먼저 확인한다.

n비트로 구별할 수 있는 비트 패턴은 2의 n제곱 개다. 부호 없는 정수의 최댓값은 이 개수보다 1 작다. 0부터 세기 때문이다. 비트 수와 표현 가능한 개수, 최대 수를 구분하면 기억장치와 카운터 문제까지 연결된다.

16진수 한 자리는 정확히 이진수 4자리와 대응한다. 이진수를 오른쪽부터 4자리씩 묶어 바꾸면 된다. BCD는 십진수 각 자리를 따로 4비트로 표현한다. 십진수 12의 순수 이진수 표현 1100과 BCD 표현 0001 0010은 다르다.

공식 1

$$0 \le x \le 2^n - 1$$

n비트 부호 없는 정수의 범위. 가능한 값의 개수는 2^n개다.

공식 2

$$1\ \text{byte} = 8\ \text{bit}$$

KiB는 1024 byte, KB는 원칙적으로 1000 byte. 시험에서 제시한 단위 정의를 우선한다.

십진수	이진수	16진수
0	0000	0
5	0101	5

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

십진수	이진수	16진수
10	1010	A
15	1111	F

함정 체크 — 여기서 틀린다

- 2^n은 최대 숫자가 아니라 패턴의 개수다.
- BCD 4비트의 1010~1111은 정상적인 한 자리 십진 숫자 표현으로 쓰지 않는다.

짧은 예제

부호 없는 6비트로 표현할 수 있는 수의 개수와 최댓값은?

1 6비트는 **2^6=64**가지 패턴이다.

2 0을 포함하므로 최댓값은 64-1이다.

답 → 64개, 최댓값 63

가리고 답하기

1010(2)는 십진수와 16진수로 각각 무엇인가?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D02

음수 표현과 2의 보수

B 선택

DAY 5 · 오전 · 첫 학습 15분 · 공식 1개

2의 보수는 정해진 비트 수에서 모든 비트를 뒤집고 1을 더해 음수를 만드는 방식이다. 먼저 비트 수를 고정해야 한다. 8비트의 0000 0101을 뒤집으면 1111 1010, 여기에 1을 더하면 1111 1011이 되어 -5를 나타낸다.

부호 있는 2의 보수의 맨 왼쪽 비트는 부호 해석에 쓰인다. 양수와 0의 표현은 보통의 이진수와 같다. 음수 범위가 양수보다 하나 더 넓고 0의 표현은 하나뿐이다. 부호 비트가 있다고 해서 최상위 비트를 떼고 나머지를 단순 절댓값으로 읽으면 틀린다.

같은 부호의 두 수를 더했는데 결과의 부호가 반대이면 부호 있는 덧셈 오버플로다. 최상위 자리의 올림만으로 부호 있는 오버플로를 판정하지 않는다. 서로 다른 부호의 두 수를 더할 때는 이런 오버플로가 발생하지 않는다.

공식 1

$$-2^{n-1} \le x \le 2^{n-1} - 1$$

n비트 2의 보수 정수 범위. 예: 8비트는 -128~127.

표현	0의 표현	n비트 범위
부호 없는 정수	하나	0 ~ 2^n-1
부호-크기 / 1의 보수	양의 0, 음의 0	-(2^(n-1)-1) ~ +(2^(n-1)-1)
2의 보수	하나	-2^(n-1) ~ 2^(n-1)-1

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

함정 체크 — 여기서 틀린다

- 비트 수가 달라지면 같은 모양의 비트열도 해석이 달라질 수 있다.
- 최상위 캐리와 부호 있는 오버플로는 같은 개념이 아니다.

짧은 예제

4비트 2의 보수에서 0111+0001은 왜 오버플로인가?

1 두 피연산자는 +7과 +1이다.

2 결과 비트 1000은 4비트 2의 보수에서 -8로 해석된다.

답 → 표현 가능한 최댓값 +7을 넘어 양수+양수가 음수가 되었기 때문이다.

가리고 답하기

8비트 2의 보수의 최소값과 최대값은?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D03 그레이코드: 인접 상태는 한 비트만 바뀐다

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 2개

반사형 그레이코드는 인접한 수 사이에서 정확히 한 비트만 변하도록 배열한 코드다. 회전 위치를 읽는 엔코더에서 여러 비트가 동시에 바뀌어 순간적으로 엉뚱한 수를 읽는 문제를 줄이는 데 유리하다. 일반 이진수의 011→100은 세 비트가 바뀌지만 그레이코드의 연속 상태는 한 비트만 바뀐다.

이진수에서 그레이코드로 바꿀 때 최상위 비트는 그대로 두고, 그 아래 각 비트는 원래 이진수의 이웃 두 비트를 XOR한다. 이전에 만든 그레이 비트와 XOR하는 것이 아니다. 반대로 그레이에서 이진수로 돌아갈 때는 이미 구한 상위 이진 비트와 현재 그레이 비트를 XOR한다.

카르노맵의 행·열 순서가 00, 01, 11, 10인 이유도 인접 칸에서 한 변수만 달라지게 하기 위해서다. 그레이코드를 보통의 자릿값으로 곧바로 더해서 십진수로 바꾸지 않는다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 카르노맵의 모서리 묶음

카르노맵: 네 모서리도 서로 인접한다

AB \ CD	00	01	11	10
00	**1**	0	0	**1**
01	0	0	0	0
11	0	0	0	0
10	**1**	0	0	**1**

모서리 4칸 = 하나의 4칸 묶음

행: B=0만 일정

열: D=0만 일정

F=(NOT B) AND (NOT D)

A와 C는 묶음 안에서 바뀐다.

상하·좌우 가장자리는 연결된다. 대각선만 닿는 칸은 인접하지 않는다.

4변수 맵의 행과 열은 00·01·11·10 순서다. 네 모서리에서는 B와 D만 0으로 고정되므로 A,C가 제거된다.

공식 1

$$G = B \oplus (B \gg 1)$$

B는 부호 없는 이진 비트열, >>1은 오른쪽 1비트 논리 시프트다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i$$

그레이 변환에서는 오른쪽 식의 두 비트가 모두 원래 이진수 비트다.

값	이진수	그레이코드
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

함정 체크 — 여기서 틀린다

- 그레이코드는 오류를 자동으로 정정하는 코드라는 뜻이 아니다.
- 변환 방향에 따라 XOR에 사용하는 상위 비트가 다르다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

짧은 예제

이진수 1011을 그레이코드로 바꾸면?

1 최상위 1은 그대로 쓴다.

2 이웃 이진 비트끼리 XOR하면 **1⊕0=1**, **0⊕1=1**, **1⊕1=0**이다.

답 → 1110

가리고 답하기

카르노맵 열을 00,01,10,11 순으로 쓰면 안 되는 이유는?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D04 논리게이트와 진리표

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 2개

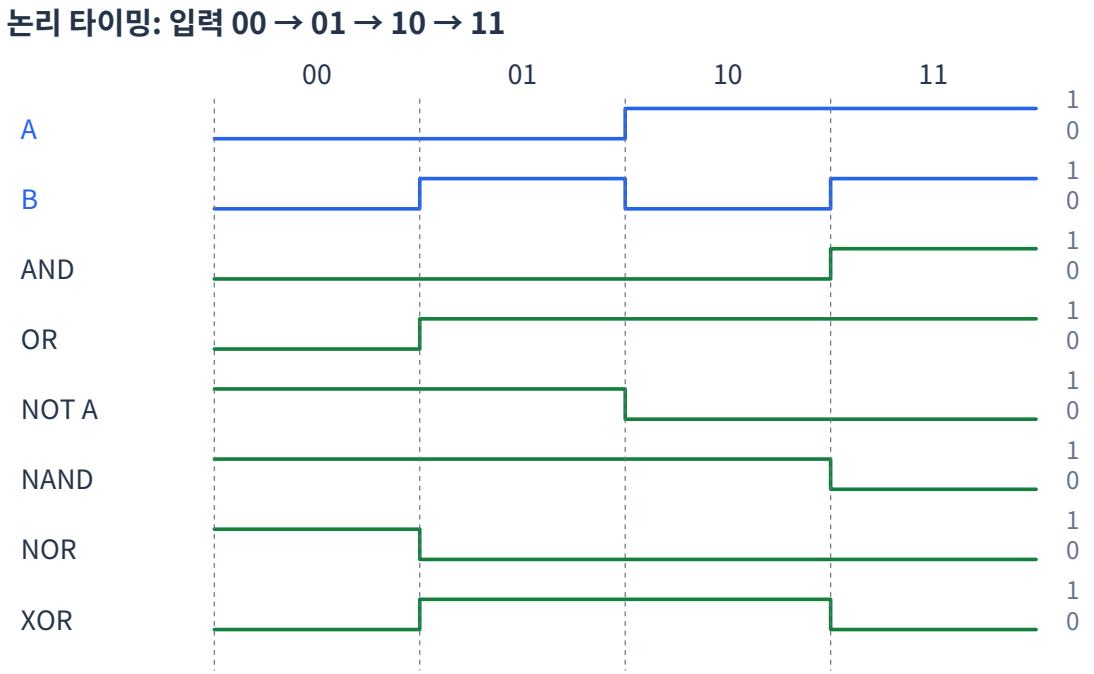
디지털 논리의 AND는 모두 1일 때만 1, OR는 하나라도 1이면 1, NOT은 입력을 뒤집는다. 논리식에서 +는 OR, 곱은 AND이므로 일반 산술과 혼동하지 않는다. 논리에서 1+1은 1이다.

2입력 XOR는 두 입력이 다르면 1, XNOR는 두 입력이 같으면 1이다. XOR의 '다르면 1' 설명은 2입력일 때의 표현이다. 여러 입력을 연쇄 XOR하면 1의 개수가 홀수일 때 1이 된다.

회로를 읽을 때 출력의 작은 동그라미는 반전을 뜻한다. 게이트의 모양을 흐릿하게 기억하기보다 입력 00,01,10,11을 넣어 네 출력을 적으면 회로식의 정오를 빠르게 확인할 수 있다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 여섯 논리함수의 타이밍



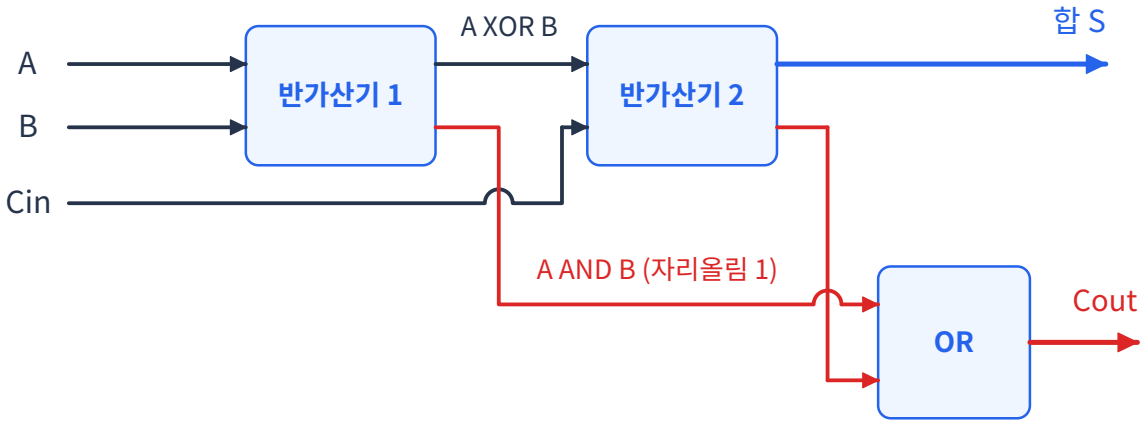
이상적인 논리 레벨만 표시한다. 실제 전파지연·접점 바운스는 생략.

같은 네 입력 조합에 대해 AND·OR·NOT·NAND·NOR·XOR의 출력을 대응시켰다. XOR은 두 입력이 서로 다를 때만 1이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

그림 · 두 반가산기로 전가산기 구성

전가산기: 반가산기 두 개와 OR 한 개



S=A XOR B XOR Cin

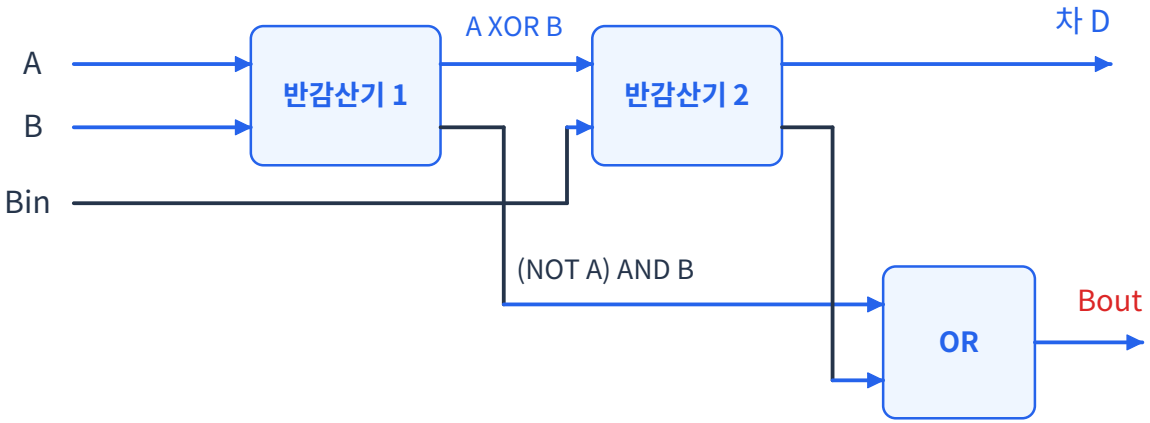
Cout=(A AND B) OR [Cin AND (A XOR B)]

반가산기 하나의 합은 XOR, 자리올림은 AND이다. 첫 합과 Cin을 두 번째 반가산기에 넣고 두 자리올림을 OR한다. 점 없는 교차 배선은 연결이 아니다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

그림 · 두 반감산기로 전감산기 구성

전감산기: A - B - Bin을 계산한다



D=A XOR B XOR Bin

Bout=[(NOT A) AND B] OR [(NOT(A XOR B)) AND Bin]

첫 단계 A-B의 차에서 다시 Bin을 뺀다. 둘 중 어느 단계라도 빌림이 생기면 Bout=1이다. 반가산기의 carry와 반감산기의 borrow는 다른 논리식이다.

공식 1

$$A \oplus B = \overline{A}B + A\overline{B}$$

2입력 XOR. **A=B**이면 0, A≠B이면 1.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$\overline{A \oplus B} = AB + \overline{A}\,\overline{B}$$

2입력 XNOR. 입력이 같은지 판단하는 기본 논리다.

입력 AB	AND / OR	XOR / XNOR	NAND / NOR
00	0 / 0	0 / 1	1 / 1
01	0 / 1	1 / 0	1 / 0
10	0 / 1	1 / 0	1 / 0
11	1 / 1	0 / 1	0 / 0

함정 체크 — 여기서 틀린다

- OR와 XOR는 입력 11에서만 출력이 다르다.
- NAND는 AND 결과 전체의 반전이다. 입력 하나만 반전하는 것이 아니다.

짧은 예제

A=1,B=1일 때 A+B, A⊕B, NAND 출력은?

1 OR는 하나라도 1이므로 1이다.

2 XOR는 입력이 같으므로 0, NAND는 **AND=1**의 반전이다.

답 → 순서대로 1, 0, 0

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

가리고 답하기

2입력 XOR가 1인 두 입력 조합은?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D05 드모르간과 범용 게이트

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 3개

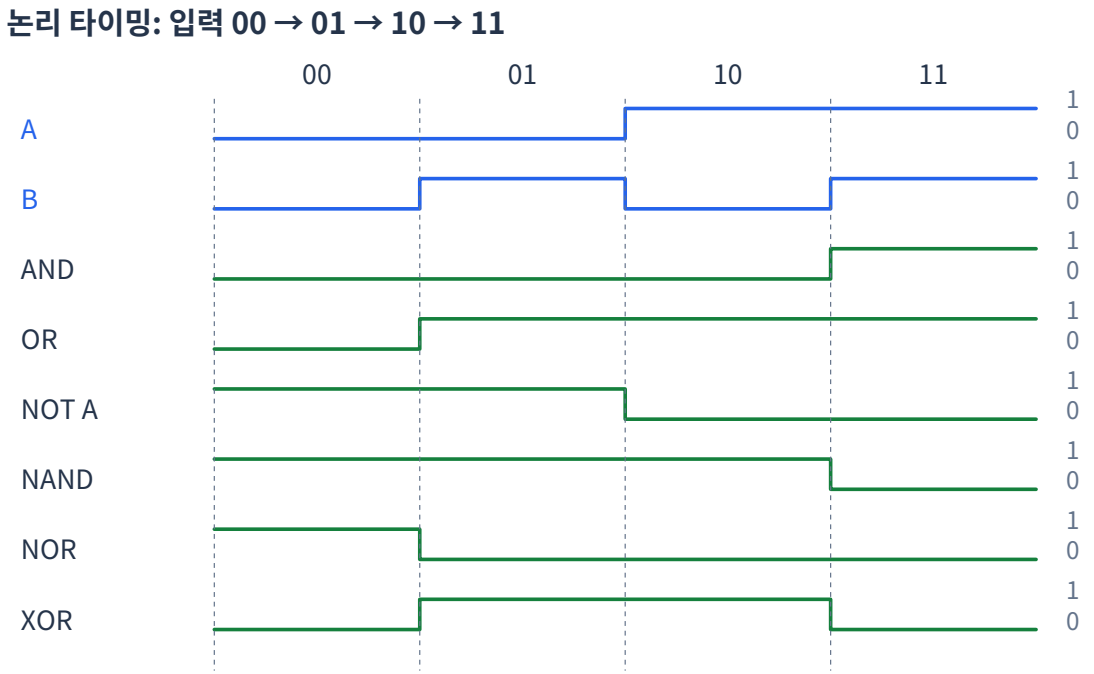
드모르간 법칙은 괄호 전체의 NOT을 안쪽으로 옮기면서 AND와 OR를 서로 바꾸고 각 입력을 반전하는 규칙이다. '기호만 바꾸기'나 '입력만 뒤집기' 중 하나만 하면 틀린다. 이중 부정은 원래 값으로 돌아온다.

NAND와 NOR는 각각 단독으로 NOT, AND, OR를 만들 수 있으므로 범용 게이트다. 단독이라는 말은 한 종류의 게이트만 반복 사용해도 된다는 뜻이며, 게이트 소자 단 한 개로 임의의 회로를 만들 수 있다는 뜻은 아니다.

NAND의 두 입력을 같은 A에 연결하면 NOT A가 된다. 먼저 NAND로 AB의 반전을 만든 뒤 다시 반전하면 AND가 된다. 입력 A와 B를 각각 반전한 다음 NAND하면 OR가 된다. NOR도 AND/OR를 맞바꾸어 같은 원리로 구성한다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 여섯 논리함수의 타이밍



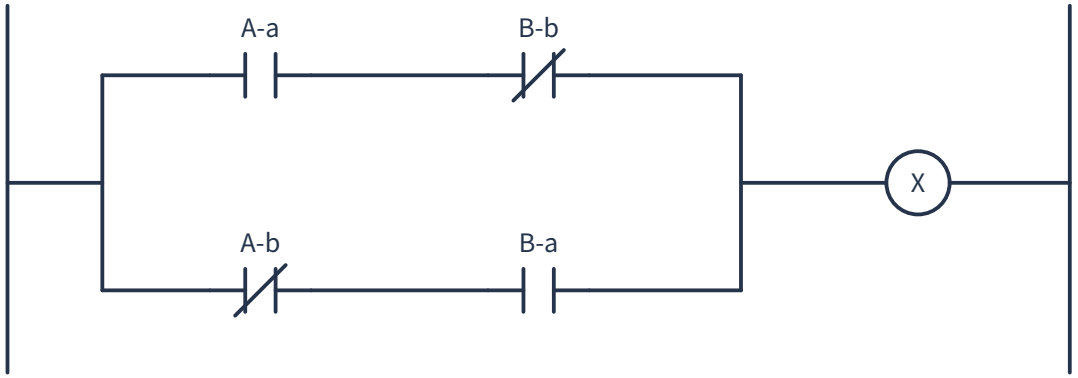
이상적인 논리 레벨만 표시한다. 실제 전파지연·접점 바운스는 생략.

같은 네 입력 조합에 대해 AND·OR·NOT·NAND·NOR·XOR의 출력을 대응시켰다. XOR은 두 입력이 서로 다를 때만 1이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

그림 · XOR의 유접점 구현

XOR 유접점 회로: 서로 다른 상태일 때만 통전



X = A · NOT B + NOT A · B

A=B=0 또는 A=B=1이면 두 분기 모두 열린다.

각 분기는 한 입력의 a접점과 다른 입력의 b접점을 직렬 연결한다. 두 분기를 병렬로 묶으면 XOR 논리가 된다.

공식 1

$$\overline{AB} = \overline{A} + \overline{B}$$

곱의 반전은 각 항을 반전한 논리합.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$\overline{A+B}=\overline{A}\,\overline{B}$$

합의 반전은 각 항을 반전한 논리곱.

공식 3

$$\overline{AA}=\overline{A}, \qquad \overline{A+A}=\overline{A}$$

NAND 또는 NOR의 두 입력을 묶으면 인버터가 된다.

함정 체크 — 여기서 틀린다

- AND 하나 또는 OR 하나만 반복해서는 NOT을 만들 수 없으므로 범용이 아니다.
- 출력 전체를 반전하는 괄호의 범위를 끝까지 확인한다.

짧은 예제

Y=(A+B)의 전체 반전일 때 AND와 NOT으로 표현하면?

1 전체 반전을 분배한다.

2 OR를 AND로 바꾸고 A,B 모두를 반전한다.

답 → **Y=NOT** A AND NOT B

가리고 답하기

범용 게이트 두 종류는? 입력 두 개를 묶으면 무엇이 되는가?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D06 불 대수·최소항과 논리식 간소화

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 3개

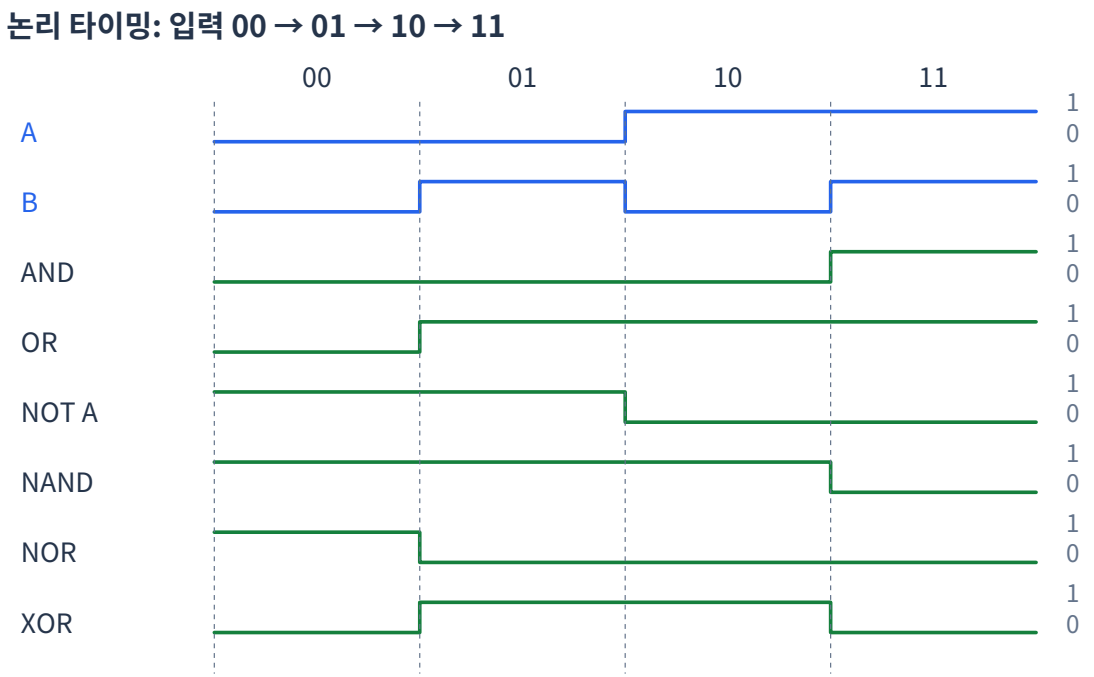
논리식 간소화는 입력별 출력은 그대로 유지하면서 게이트 수나 식의 길이를 줄이는 작업이다. **A+AB=A**는 A가 1이면 이미 결과가 1이고, A가 0이면 AB도 0이기 때문에 성립한다. 암기한 식이 생각나지 않으면 **A=0**,1 두 경우로 확인한다.

최소항은 모든 변수가 한 번씩 들어간 AND 항이다. 예를 들어 **ABC=101**에서만 1인 최소항은 A·NOT B·C 다. 합의 최소항 표기 Σm(1,3)은 해당 번호의 입력에서 출력이 1이라는 뜻이다. 변수의 비트 순서를 A가 최상위라고 정했는지 먼저 확인한다.

흡수법칙, 보수법칙, 분배법칙만으로도 짧은 기출 논리식을 많이 정리할 수 있다. 간소화가 끝난 뒤에는 진리표의 몇 조합을 대입하여 원래 식과 같은지 점검한다. 가산기에서의 산술 덧셈과 불 대수의 OR를 섞지 않는다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 여섯 논리함수의 타이밍



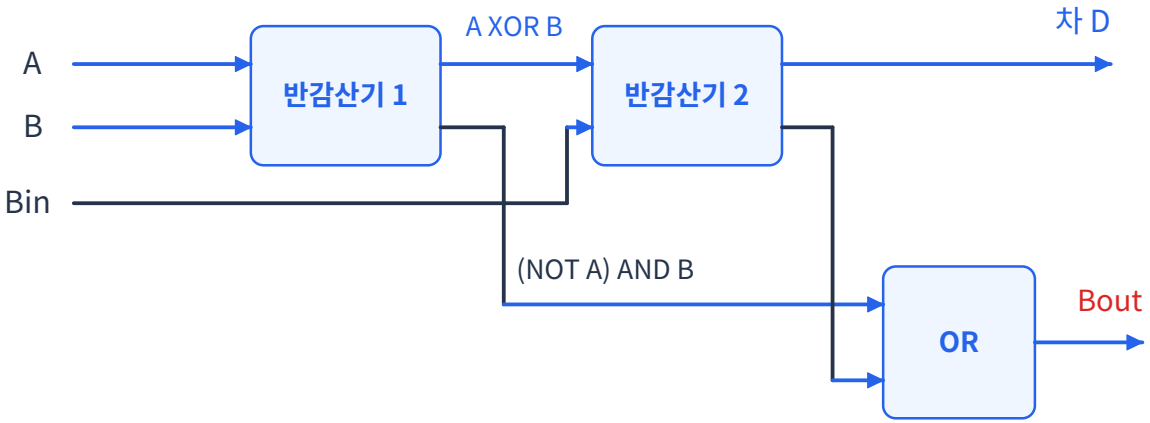
이상적인 논리 레벨만 표시한다. 실제 전파지연·접점 바운스는 생략.

같은 네 입력 조합에 대해 AND·OR·NOT·NAND·NOR·XOR의 출력을 대응시켰다. XOR은 두 입력이 서로 다를 때만 1이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

그림 · 두 반감산기로 전감산기 구성

전감산기: A - B - Bin을 계산한다



D=A XOR B XOR Bin

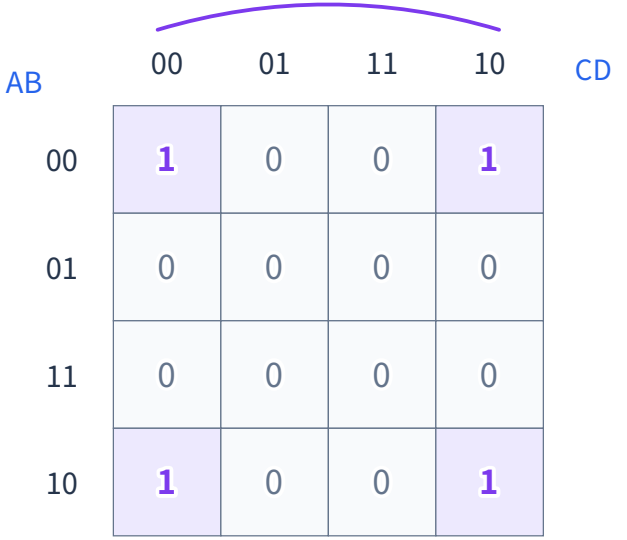
Bout=[(NOT A) AND B] OR [(NOT(A XOR B)) AND Bin]

첫 단계 A-B의 차에서 다시 Bin을 뺀다. 둘 중 어느 단계라도 빌림이 생기면 Bout=1이다. 반가산기의 carry와 반감산기의 borrow는 다른 논리식이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 카르노맵의 모서리 묶음

카르노맵: 네 모서리도 서로 인접한다



모서리 4칸 = 하나의 4칸 묶음

행: B=0만 일정

열: D=0만 일정

F=(NOT B) AND (NOT D)

A와 C는 묶음 안에서 바뀐다.

상하·좌우 가장자리는 연결된다. 대각선만 닿는 칸은 인접하지 않는다.

4변수 맵의 행과 열은 00·01·11·10 순서다. 네 모서리에서는 B와 D만 0으로 고정되므로 A,C가 제거된다.

공식 1

$$A + AB = A, \qquad A(A + B) = A$$

흡수법칙.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$A + \overline{A}B = A + B$$

A=0일 때 B, **A=1**일 때 1인 함수다.

공식 3

$$AB + A\overline{B} = A$$

B+NOT **B=1**로 묶는다.

기본 규칙	OR	AND
같은 항	A+A=A	AA=A
보수	A+NOT A=1	A · NOT A=0
항등 / 지배	A+0=A / A+1=1	A · 1=A / A · 0=0

함정 체크 — 여기서 틀린다

- 불 대수에서 **A+A=2A**로 쓰지 않는다.
- 최소항 번호는 행·열의 위치가 아니라 변수 비트열을 십진수로 읽은 값이다.

짧은 예제

AB+NOT A · B를 간소화하면?

1 공통항 B를 묶어 (A+NOT A)B로 쓴다.

2 괄호는 1이다.

답 → B

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

가리고 답하기

ABC의 순서에서 최소항 m5를 쓰면?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D07 카르노맵: 묶는 규칙과 남는 변수

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 1개

카르노맵은 한 비트만 다른 최소항을 이웃하게 놓은 표다. 가로·세로 순서는 00,01,11,10이다. SOP, 즉 곱의 합 형태를 만들 때 1을 1,2,4,8개처럼 2의 거듭제곱 개수로 직사각형 묶는다. 큰 묶음일수록 없어지는 변수가 많다.

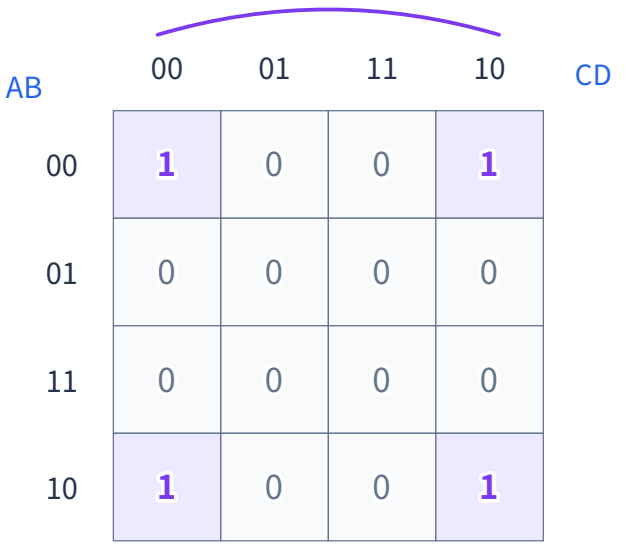
서로 겹치는 묶음을 써도 된다. 왼쪽과 오른쪽 끝, 위와 아래 끝도 서로 이웃이다. 대각선만 닿는 칸은 이웃이 아니다. 필요한 모든 1을 덮어야 하며, 실제 0이 들어간 묶음을 만들면 안 된다. 묶음에서 값이 바뀌는 변수는 지우고 끝까지 일정한 변수만 식에 남긴다.

무관항 X는 간소화에 유리할 때만 1로 포함하고 나머지는 사용하지 않아도 된다. 모든 X를 반드시 1로 바꾸는 것이 아니다. POS, 즉 합의 곱을 구하라는 문제는 0을 묶는 방식이므로 요구 형태를 먼저 읽는다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

그림 · 카르노맵의 모서리 묶음

카르노맵: 네 모서리도 서로 인접한다



모서리 4칸 = 하나의 4칸 묶음

행: B=0만 일정

열: D=0만 일정

F=(NOT B) AND (NOT D)

A와 C는 묶음 안에서 바뀐다.

상하·좌우 가장자리는 연결된다. 대각선만 닿는 칸은 인접하지 않는다.

4변수 맵의 행과 열은 00·01·11·10 순서다. 네 모서리에서는 B와 D만 0으로 고정되므로 A,C가 제거된다.

공식 1

$$F(A,B,C)=\sum m(0,1,4,5)=\overline{B}$$

ABC 순서에서 **B=0**인 네 칸을 묶은 예. A와 C는 변하므로 사라진다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

A / BC	00	01	11	10
0	m0	m1	m3	m2
1	m4	m5	m7	m6

함정 체크 — 여기서 틀린다

- 3칸·6칸 묶음이나 L자 모양 묶음은 허용되지 않는다.
- 일부 해설의 위치표의 **AB=11,CD=10**은 m16이 아니라 m14다. 4변수 번호는 0~15다.

짧은 예제

3변수 카르노맵에서 1이 m2,m3,m6,m7에만 있다. 최소식은?

1 번호를 ABC로 바꾸면 010,011,110,111이다.

2 모두 **B=1**이며 A,C는 바뀐다.

답 → **F=B**

가리고 답하기

카르노맵의 네 모서리 칸을 하나의 묶음으로 볼 수 있는가?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D08

반가산기와 전가산기

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 3개

반가산기는 두 1비트 수 A,B를 더해 합 S와 자리올림 C를 만든다. 합의 한 자리 값은 XOR, 자리올림은 AND다. **A=B=1**이면 산술 합 2가 이진수 10이므로 **S=0**,**C=1**이다.

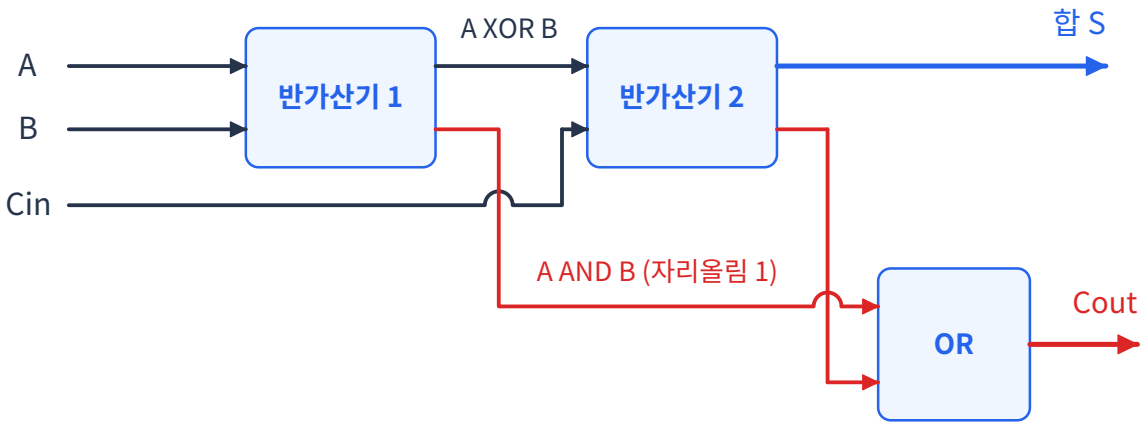
전가산기는 A,B뿐 아니라 아래 자리에서 넘어온 입력 캐리도 더한다. 세 입력 중 1이 홀수 개이면 합 비트가 1이고, 1이 두 개 이상이면 출력 캐리가 1이다. 여러 비트 덧셈기는 전가산기를 자리별로 이어 구성한다.

반가산기는 입력 캐리를 처리하지 못한다. 이름의 '반'은 두 출력을 절반만 만든다는 뜻이 아니다. 반가산기 두 개와 OR 게이트를 조합하면 전가산기를 구성할 수 있다. 식을 통째로 기억하되 진리표 의미로 다시 만들 수 있게 한다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 두 반가산기로 전가산기 구성

전가산기: 반가산기 두 개와 OR 한 개



S=A XOR B XOR Cin

Cout=(A AND B) OR [Cin AND (A XOR B)]

반가산기 하나의 합은 XOR, 자리올림은 AND이다. 첫 합과 Cin을 두 번째 반가산기에 넣고 두 자리올림을 OR한다. 점 없는 교차 배선은 연결이 아니다.

공식 1

$$S = A \oplus B, \qquad C = AB$$

반가산기. C는 출력 자리올림.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$S = A \oplus B \oplus C_{\text{in}}$$

전가산기의 합.

공식 3

$$C_{\text{out}} = AB + C_{\text{in}}(A \oplus B)$$

전가산기 캐리. AB+ACin+BCin과 동치다.

A B	합 S	캐리 C
0 0	0	0
0 1	1	0
1 0	1	0
1 1	0	1

함정 체크 — 여기서 틀린다

- 가산기 합은 OR가 아니라 XOR다.
- 입력 캐리와 출력 캐리는 다른 신호다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

짧은 예제

전가산기에 **A=1,B=0,Cin=1**이 들어오면?

1 산술 합은 **2=10(2)**다.

2 합 비트는 0, 다음 자리로 넘기는 캐리는 1이다.

답 → **S=0, Cout=1**

가리고 답하기

반가산기와 전가산기의 입력 개수 차이는?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D09 반감산기와 전감산기

A 필수

DAY 5 · 오전 · 첫 학습 20분 · 공식 3개

반감산기는 A에서 B를 빼고 차 D와 빌림 Bout을 출력한다. **A=0,B=1**이면 이 자리에서 1을 뺄 수 없어 위 자리에서 2를 빌려 **2-1=1**로 계산한다. 그래서 차는 1, 빌림은 1이다.

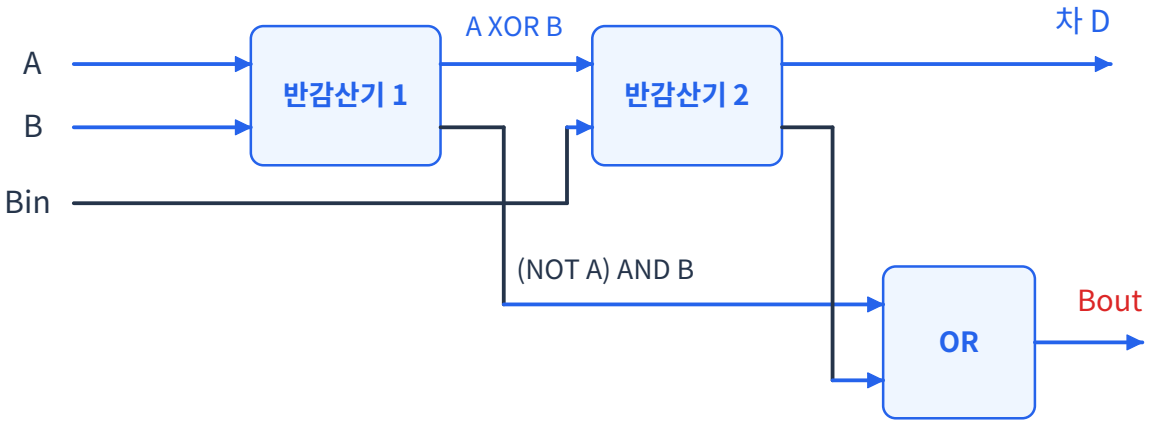
전감산기는 아래 자리에서 요구한 입력 빌림 Bin까지 포함하여 A-B-Bin을 수행한다. 차는 세 입력의 XOR다. 빌림은 A가 부족한 경우에 1이 된다. **A=0**일 때 B 또는 Bin이 1이면 빌림이 필요하고, **A=1**이어도 **B=Bin=1**이면 빌림이 필요하다.

가산기와 감산기는 합 또는 차의 XOR 구조가 비슷하지만 캐리와 빌림 식은 다르다. 특히 반감산기의 빌림은 NOT A · B이며 AB가 아니다. 빌림을 잘못 외웠을 때 **A=0,B=1**이라는 대표 조합을 대입하면 즉시 걸러낼 수 있다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

그림 · 두 반감산기로 전감산기 구성

전감산기: A - B - Bin을 계산한다



D=A XOR B XOR Bin

Bout=[(NOT A) AND B] OR [(NOT(A XOR B)) AND Bin]

첫 단계 A-B의 차에서 다시 Bin을 뺀다. 둘 중 어느 단계라도 빌림이 생기면 Bout=1이다. 반가산기의 carry와 반감산기의 borrow는 다른 논리식이다.

공식 1

$$D = A \oplus B, \qquad B_{\text{out}} = \overline{A}B$$

반감산기. A가 피감수, B가 감수다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$D = A \oplus B \oplus B_{\text{in}}$$

전감산기의 차.

공식 3

$$B_{\text{out}} = \overline{A}B + \overline{A}B_{\text{in}} + BB_{\text{in}}$$

전감산기의 빌림. 입력 순서 A-B-Bin을 유지한다.

A B Bin	차 D	빌림 Bout
0 0 0	0	0
0 0 1	1	1
0 1 0	1	1
0 1 1	0	1
1 0 0	1	0
1 0 1	0	0
1 1 0	0	0
1 1 1	1	1

함정 체크 — 여기서 틀린다

- A와 B를 바꾸면 차 비트는 같을 수 있지만 빌림은 달라진다.
- 출력 빌림은 위 자리에서 2를 빌려왔다는 표시이며 음수 부호 비트와 다르다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

짧은 예제

전감산기 입력 **A=0,B=1,Bin=1**일 때 출력은?

1 **0-1-1=-2**이므로 위 자리에서 2를 빌린다.

2 이 자리 차는 **2-1-1=0**이다.

답 → **D=0, Bout=1**

가리고 답하기

반감산기에서 **A=1,B=0**이면 빌림이 필요한가?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D10 선택기·해독기·부호기 구분

B 선택

DAY 5 · 오전 · 첫 학습 15분 · 공식 2개

멀티플렉서(MUX)는 여러 데이터 입력 중 하나를 선택해 하나의 출력으로 보낸다. 선택 신호가 주소처럼 어느 입력을 내보낼지 정한다. 디멀티플렉서(DEMUX)는 한 입력을 여러 출력 중 선택된 곳으로 보낸다.

디코더는 n비트 코드에 대응하는 최대 2^n개의 출력 중 하나를 활성화한다. 인코더는 반대로 활성 입력을 코드로 바꾼다. 여러 입력이 동시에 활성화될 수 있다면 우선순위를 정하는 우선순위 인코더가 필요하다.

실제 회로는 활성 레벨이 0인 경우도 있다. 이 자료의 기본 설명은 활성 1을 기준으로 한다. 문제에서 EN, enable, 출력의 반전 표시를 주면 그 조건을 우선한다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 1

$$Y = \overline{S}I_0 + SI_1$$

2:1 MUX. **S=0**이면 I0, **S=1**이면 I1.

공식 2

$$N = 2^k$$

입력이 2^k개인 MUX는 일반적으로 k개의 선택 비트가 필요하다.

소자	주된 역할	기억할 방향
MUX	입력 선택	여러 입력 → 한 출력
DEMUX	출력 경로 선택	한 입력 → 여러 출력 중 하나
디코더	코드 해독	n비트 → 최대 2^n선
인코더	코드 생성	입력 위치 → 이진 코드

함정 체크 — 여기서 틀린다

- 데이터 입력 개수와 선택 입력 개수를 혼동하지 않는다.
- 일반 인코더는 여러 활성 입력이 동시에 주어지면 모호할 수 있다.

짧은 예제

8:1 MUX의 선택 입력 수는?

1 **2^k=8**을 만족하는 k를 찾는다.

답 → 3개

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

가리고 답하기

한 입력을 선택된 한 출력 경로로 보내는 것은?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D11 래치·플립플롭과 다음 상태

A 필수

DAY 5 · 오후 · 첫 학습 20분 · 공식 3개

조합회로 출력은 현재 입력만으로 정해지고, 순서회로 출력은 현재 입력과 저장된 상태에 의존한다. 플립플롭 한 개는 1비트를 저장한다. 래치는 일반적으로 특정 클럭 레벨 동안 입력에 반응하고, 에지 트리거 플립플롭은 지정된 상승 또는 하강 순간에 상태를 갱신한다.

D 플립플롭은 지정된 클럭 순간의 D를 다음 Q로 저장한다. T는 **T=1**이면 반전, **T=0**이면 유지한다. JK는 00 유지, 01 리셋, 10 세트, 11 반전이다. JK의 11을 금지 상태로 외우면 안 된다.

SR의 금지 조합은 회로의 활성 레벨에 달린다. 활성 1의 NOR형 SR 래치는 **S=R=1**이 금지 조합이고, 활성 0의 NAND형은 두 입력이 모두 0일 때 금지 조합이다. 비동기 preset/clear가 주어지면 클럭보다 해당 제어가 우선할 수 있다.

공식 1

$$Q_{n+1} = D$$

D 플립플롭의 특성식. 지정된 활성 에지에서 갱신한다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$Q_{n+1} = T \oplus Q_n$$

T 플립플롭. **T=1**이면 반전.

공식 3

$$Q_{n+1} = J\overline{Q_n} + \overline{K}Q_n$$

JK 플립플롭의 특성식.

J	K	다음 상태
0	0	유지 Q
0	1	0 (Reset)
1	0	1 (Set)
1	1	반전 NOT Q

함정 체크 — 여기서 틀린다

- 래치와 에지 트리거 플립플롭의 반응 시간을 구별한다.
- 순서회로에는 초기 상태와 클럭 조건이 필요하다.

짧은 예제

Q=1인 JK 플립플롭에 **J=K=1**을 주고 활성 클럭이 한 번 오면?

1 **JK=11**은 반전이다.

답 → 다음 **Q=0**

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

가리고 답하기

JK=00은 어떤 동작인가?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D12 카운터·시프트 레지스터

B 선택

DAY 5 · 오후 · 첫 학습 15분 · 공식 1개

n개의 플립플롭으로 구성된 이진 카운터는 최대 2^n개 상태를 표현한다. MOD-N 카운터는 N개 상태를 반복하며 필요한 최소 플립플롭 수는 2^n이 N 이상이 되는 최소 정수 n이다. 초기 상태와 세는 방향을 먼저 읽는다.

비동기 리플 카운터는 앞단 출력이 다음 단 클럭을 구동하여 단계별 전파 지연이 생긴다. 동기 카운터는 플립플롭들이 공통 클럭을 받는다. **T=1** 또는 **JK=11**인 플립플롭은 클럭마다 반전해 주파수를 2분주한다.

논리 오른쪽 시프트는 왼쪽을 0으로 채운다. 부호 없는 수에서는 한 번 시프트할 때 2로 나눈 몫과 같다. 산술 오른쪽 시프트는 부호를 유지하기 위해 최상위 부호 비트를 채운다. 왼쪽 시프트가 항상 정확한 2배라는 말은 넘침이 없을 때만 맞다.

공식 1

$$N_{\max} = 2^n, \qquad f_{\text{out}} = \frac{f_{\text{clk}}}{2^n}$$

2진 분주 카운터의 최상위 단계 기준. MOD-N 전체 반복 주기는 N개의 클럭이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

함정 체크 — 여기서 틀린다

- MOD-10은 10비트가 아니라 최소 4비트가 필요하다.
- 초기값이 000이 아닐 수 있다. 리셋 조건과 활성 에지를 확인한다.

짧은 예제

초기값 000인 3비트 증가 카운터에 클럭 10회가 들어오면?

1 8개 상태마다 처음으로 돌아간다.

2 10을 8로 나눈 나머지는 2다.

답 → 010

가리고 답하기

부호 없는 00110000을 논리 오른쪽으로 세 번 옮기면?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D13 CMOS 게이트를 스위치로 읽기

B 선택

DAY 5 · 오후 · 첫 학습 15분 · 공식 0개

기본 CMOS 논리에서는 PMOS가 위쪽 전원으로 당기는 경로, NMOS가 아래쪽 접지로 내리는 경로를 만든다. 이상적인 스위치 모델에서 NMOS는 게이트가 1일 때 켜지고 PMOS는 0일 때 켜진다.

CMOS NAND는 NMOS 두 개가 직렬이므로 두 입력이 모두 1이어야 출력이 접지로 내려간다. PMOS는 병렬로 연결되어 입력 중 하나가 0이면 출력을 전원 쪽으로 올린다. CMOS NOR는 그 반대로 NMOS 병렬, PMOS 직렬이다.

정상 논리 상태에서는 정적 전력이 작다는 것이 CMOS의 장점이다. 실제 소자에는 누설전류가 있고 스위칭할 때는 충방전 전력이 소모되므로 전력이 항상 정확히 0이라고 외우지 않는다. 회로 그림의 소자 연결을 볼 수 있다면 직렬/병렬 경로부터 추적한다.

게이트	NMOS 풀다운	PMOS 풀업
NOT	NMOS 1개	PMOS 1개
2입력 NAND	직렬	병렬
2입력 NOR	병렬	직렬

함정 체크 — 여기서 틀린다

- NMOS와 PMOS의 ON 조건을 같은 것으로 외우지 않는다.
- 논리게이트 회로와 아날로그 MOSFET 증폭기 동작 영역을 섞지 않는다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

짧은 예제

NMOS가 병렬인 기본 2입력 CMOS 게이트는?

1 입력 중 하나만 1이어도 출력이 접지로 내려간다.

답 → NOR

가리고 답하기

CMOS NAND에서 **A=B=1**이면 출력이 내려가는 이유는?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D14 폰 노이만 구조와 컴퓨터의 구성

A 필수

DAY 5 · 오후 · 첫 학습 20분 · 공식 0개

폰 노이만 구조의 핵심은 명령어와 데이터를 기억장치에 저장하는 프로그램 내장 방식이다. CPU가 명령어를 가져와 해석하고 실행한다. 전통적인 설명에서는 명령어와 데이터가 같은 기억공간과 전송 경로를 공유한다.

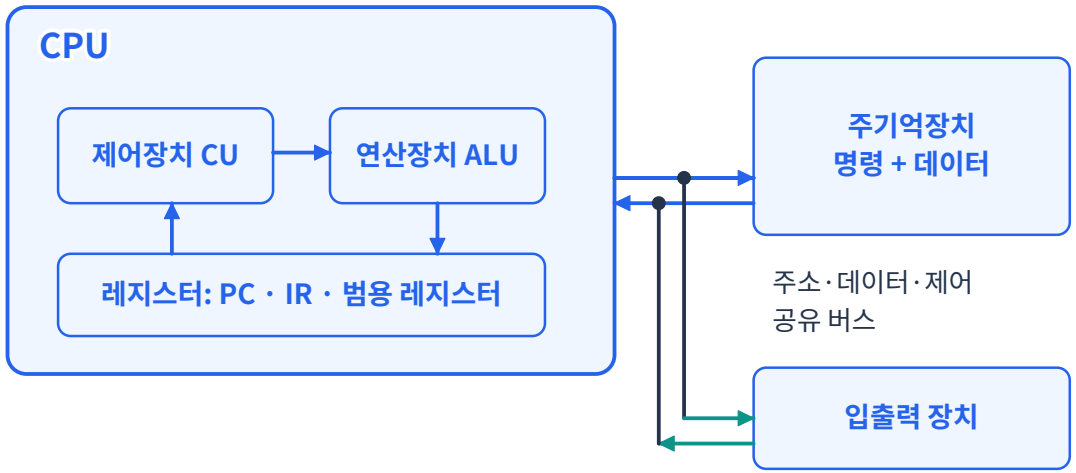
하버드 구조는 명령어와 데이터의 기억공간 또는 경로를 분리한다. 폰 노이만 병목은 CPU와 기억장치 사이의 전송 능력이 전체 성능을 제한하는 현상을 말한다. 현대 프로세서는 명령어 캐시와 데이터 캐시를 따로 두는 등 혼합 구조를 쓰므로 모든 실제 컴퓨터가 교과서 그림 그대로라고 단정하지 않는다.

CPU의 기본 구성은 연산·논리 장치(ALU), 제어장치, 레지스터다. 주기억장치와 입출력장치는 CPU와 구분한다. 기본적인 명령 실행 흐름을 순차적으로 배우되 분기·인터럽트가 있으면 다음 명령 위치가 바뀔 수 있다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

그림 · 컴퓨터의 기본 구성과 명령 실행

폰 노이만 구조: 명령과 데이터를 같은 메모리에서 읽는다



명령 인출 → 해독 → 피연산자 읽기 → 실행 → 결과 저장

PC: 다음 명령 주소 · IR: 현재 명령 · ALU: 산술·논리 연산

이 도식은 공유 메모리의 논리 구조를 단순화한다. 실제 구현에서는 캐시·버스·입출력 제어기가 사이에 놓일 수 있다.

구성요소	주요 기능	구분 포인트
ALU	산술·논리 연산	제어 신호 생성 장치와 구별
제어장치	명령 해독·동작 제어	저장공간 자체가 아님
레지스터	CPU 내부의 빠른 임시 저장	캐시·RAM과 구별

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

구성요소	주요 기능	구분 포인트
주기억장치	실행할 프로그램과 데이터 보관	CPU 밖의 큰 작업공간

함정 체크 — 여기서 틀린다

- 폰 노이만 구조를 오직 한 명령만 처리할 수 있는 모든 현대 구현이라는 뜻으로 확대하지 않는다.
- CPU와 컴퓨터 전체를 같은 구성요소로 보지 않는다.

짧은 예제

명령어와 데이터를 한 기억공간에 저장하는 기본 구조의 이름은?

1 프로그램 내장 방식과 기억공간 공유라는 두 단서를 연결한다.

답 → 폰 노이만 구조

가리고 답하기

명령어를 해독하고 각 장치에 동작 신호를 보내는 CPU 구성요소는?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D15 명령어: opcode와 operand

A 필수

DAY 5 · 오후 · 첫 학습 20분 · 공식 0개

명령어의 opcode는 무엇을 할지, operand는 그 연산의 대상이 무엇인지 나타낸다. ADD R1,R2라는 표기에서 ADD는 덧셈 동작이고 R1,R2는 연산 대상 레지스터를 가리킨다. 구체적인 결과 저장 위치와 표기 순서는 명령어 체계마다 다를 수 있다.

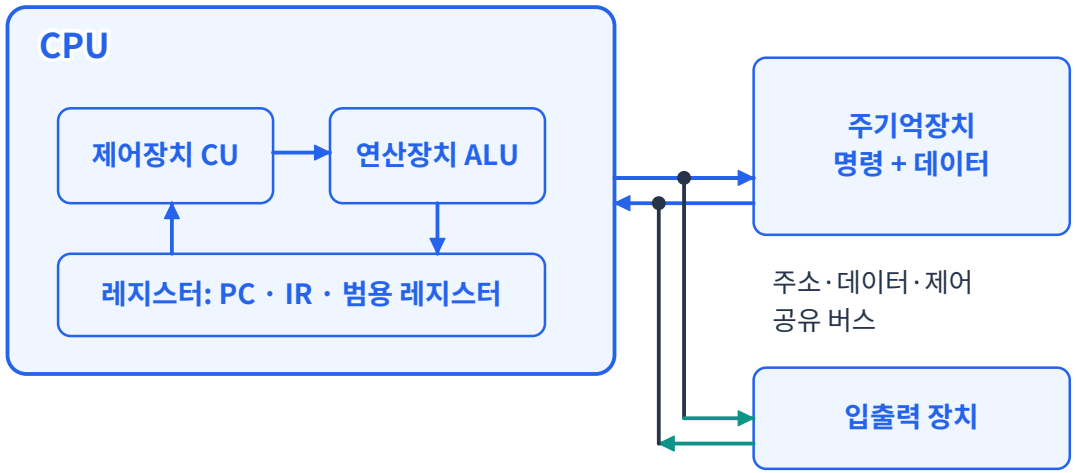
operand는 항상 실제 숫자 데이터인 것이 아니다. 즉시값, 레지스터 번호, 기억장치 주소처럼 데이터의 위치나 값을 지정할 수 있다. 따라서 'operand는 무조건 주소'와 'operand는 무조건 데이터 값'은 둘 다 지나친 일반화다.

즉시 주소지정은 명령 안의 상수 자체가 데이터이고, 직접 주소지정은 주소 필드가 유효주소다. 간접 주소지정은 그 주소에 들어 있는 값을 다시 주소로 사용한다. 레지스터 주소지정은 해당 레지스터에 값이 있다. 주소와 그 주소에 저장된 값을 구별하는 것이 핵심이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 컴퓨터의 기본 구성과 명령 실행

폰 노이만 구조: 명령과 데이터를 같은 메모리에서 읽는다



명령 인출 → 해독 → 피연산자 읽기 → 실행 → 결과 저장

PC: 다음 명령 주소 · IR: 현재 명령 · ALU: 산술·논리 연산

이 도식은 공유 메모리의 논리 구조를 단순화한다. 실제 구현에서는 캐시·버스·입출력 제어기가 사이에 놓일 수 있다.

방식	명령의 필드가 뜻하는 것	개념 예
즉시	데이터 값 자체	상수 5를 사용
직접	데이터의 기억장치 주소	M[100] 사용
간접	실제 주소를 가진 장소	M[M[100]] 사용

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

방식	명령의 필드가 뜻하는 것	개념 예
레지스터	데이터가 든 레지스터	R1의 값 사용

함정 체크 — 여기서 틀린다

- operand와 opcode를 뒤집어 외우지 않는다.
- 주소 100과 그 주소에 저장된 값 M[100]은 다르다.

짧은 예제

명령의 주소 필드가 100, **M[100]=300**, **M[300]=7**이다. 직접·간접 방식에서 읽는 값은?

1 직접이면 M[100]을 읽는다.

2 간접이면 **M[100]=300**을 주소로 하여 M[300]을 읽는다.

답 → 직접 300, 간접 7

가리고 답하기

operand가 실제 값일 수도 있는 대표 주소지정 방식은?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D16 레지스터와 명령 실행 순서

A 필수

DAY 5 · 오후 · 첫 학습 20분 · 공식 0개

PC는 다음에 가져올 명령의 주소를, IR은 현재 해독·실행할 명령 자체를 보관한다. MAR는 기억장치 접근 주소를, MDR 또는 MBR는 기억장치와 주고받는 데이터를 보관한다. 이름이 비슷해도 주소와 내용이라는 구분이 중요하다.

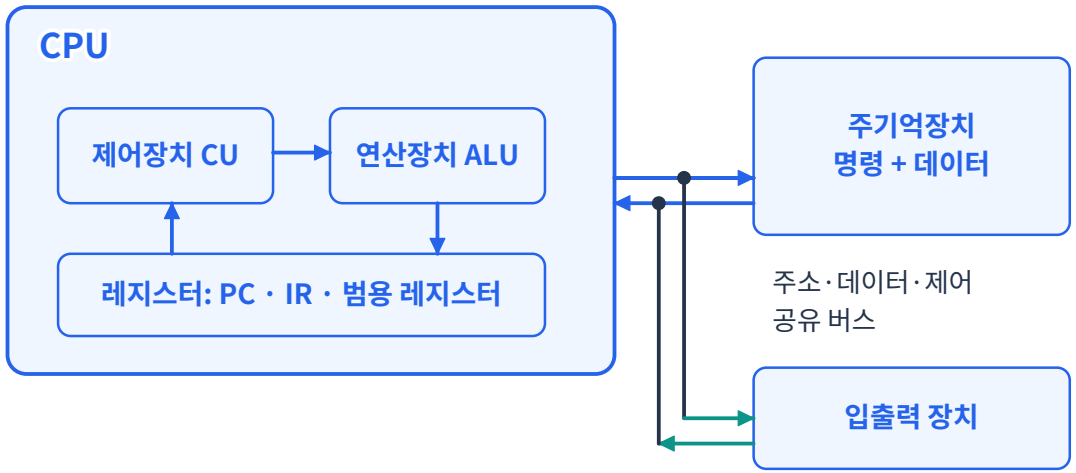
기본 명령 주기는 인출(fetch), 해독(decode), 실행(execute)으로 설명한다. 인출 때 PC가 가리키는 주소의 명령을 읽어 IR로 가져온다. 순차 실행이면 PC는 다음 명령 위치로 갱신되지만 증가량은 명령 길이와 주소 단위에 달려 무조건 1이라고 할 수 없다.

분기 명령이나 인터럽트는 PC의 흐름을 바꾼다. 누산기(ACC)는 연산의 중간 결과를 담고, 상태 레지스터에는 영·부호·캐리·오버플로 같은 상태가 기록될 수 있다. 상태 비트의 실제 구성은 CPU마다 다르다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 컴퓨터의 기본 구성과 명령 실행

폰 노이만 구조: 명령과 데이터를 같은 메모리에서 읽는다



명령 인출 → 해독 → 피연산자 읽기 → 실행 → 결과 저장

PC: 다음 명령 주소 · IR: 현재 명령 · ALU: 산술·논리 연산

이 도식은 공유 메모리의 논리 구조를 단순화한다. 실제 구현에서는 캐시·버스·입출력 제어기가 사이에 놓일 수 있다.

이름	보관 내용	암기 대조
PC	다음 인출 명령의 주소	주소
IR	현재 명령	명령 내용
MAR	접근할 메모리 주소	주소

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

이름	보관 내용	암기 대조
MDR/MBR	메모리 전송 데이터	데이터

함정 체크 — 여기서 틀린다

- PC가 다음 명령의 내용 자체를 저장한다고 하지 않는다.
- 명령어 인출과 피연산자 인출은 같은 종류의 정보가 아니다.

짧은 예제

PC=200이고 주소 200에 ADD 명령이 있다. 그 명령을 보관하여 해독하는 레지스터는?

1 PC는 위치를 알려준다.

2 그 위치에서 읽은 명령 내용은 IR로 전달된다.

답 → IR

가리고 답하기

기억장치와 주고받는 데이터가 잠시 저장되는 레지스터의 두 흔한 이름은?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D17

기억장치 계층과 RAM·ROM

A 필수

DAY 5 · 오후 · 첫 학습 20분 · 공식 0개

기억장치는 대체로 레지스터, 캐시, 주기억장치, 보조기억장치 순으로 내려갈수록 용량이 크고 비트당 비용이 낮으며 접근은 느려진다. 빠른 저장장치를 큰 용량으로 전부 만드는 것이 비싸기 때문에 계층 구조를 사용한다.

SRAM과 DRAM은 모두 일반적으로 휘발성이다. SRAM은 상태 유지에 주기적인 refresh가 필요 없고 빠르지만 비싸며, DRAM은 커패시터 전하를 이용하여 refresh가 필요하고 집적도가 높다. 캐시는 보통 SRAM, 주기억장치는 보통 DRAM으로 구현한다.

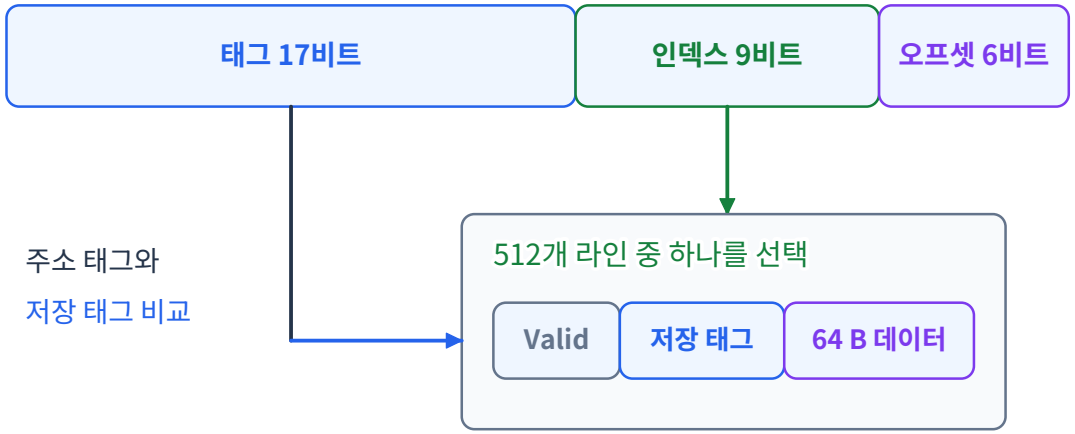
ROM 계열과 플래시 메모리는 전원이 없어도 내용을 유지하는 비휘발성 저장이다. ROM이라는 분류를 '어떠한 방식으로도 다시 기록할 수 없다'고 일반화하면 안 된다. EPROM, EEPROM, 플래시는 각자의 방법으로 지우거나 다시 쓸 수 있다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 캐시 주소 분해와 적중 판정

직접 사상 캐시: 태그·인덱스·블록 내 위치

예: 32비트 주소 · 캐시 데이터 32 KiB · 블록 64 B



라인=메모리 블록 번호 mod 512 · Valid=1이고 태그가 같으면 적중

데이터 용량만 32 KiB로 가정한 직접 사상 예이다. 64 B 오프셋은 6비트, 512라인 인덱스는 9비트이며 나머지 17비트가 태그다.

기억장치	refresh	보통의 특징
SRAM	불필요	휘발성, 빠름, 캐시에 사용
DRAM	필요	휘발성, 주기억장치에 사용
ROM 계열	DRAM식 refresh 불필요	비휘발성, 종류별 기록·소거 방식 다름

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

기억장치	refresh	보통의 특징
플래시	DRAM식 refresh 불필요	비휘발성, 블록 단위 소거

함정 체크 — 여기서 틀린다

- 정적 RAM의 정적은 비휘발성이라는 뜻이 아니다.
- 캐시와 가상기억장치는 다른 개념이다.

짧은 예제

전원을 끄면 사라지지만 refresh는 필요 없는 메모리는?

1 휘발성이므로 ROM·플래시를 제외한다.

2 DRAM은 refresh가 필요하다.

답 → SRAM

가리고 답하기

DRAM이 refresh를 필요로 하는 이유는?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D18 캐시: 지역성·적중률·쓰기 정책

A 필수

DAY 5 · 오후 · 첫 학습 20분 · 공식 2개

캐시는 CPU가 최근 사용했거나 곧 사용할 가능성이 큰 명령·데이터를 가까운 빠른 기억장치에 보관한다. 같은 것을 곧 다시 쓰는 성질은 시간적 지역성, 가까운 주소를 이어 쓰는 성질은 공간적 지역성이다. 반복문 변수 재사용과 배열 순회로 각각 기억한다.

원하는 데이터가 캐시에 있으면 hit, 없으면 miss다. 적중률은 hit 횟수/전체 접근 횟수다. 평균 접근시간은 보통 hit 시간에 miss율×추가 miss penalty를 더한다. 문제에서 miss 시간이 총시간인지 추가 지연인지에 따라 같은 숫자를 두 번 더하지 않아야 한다.

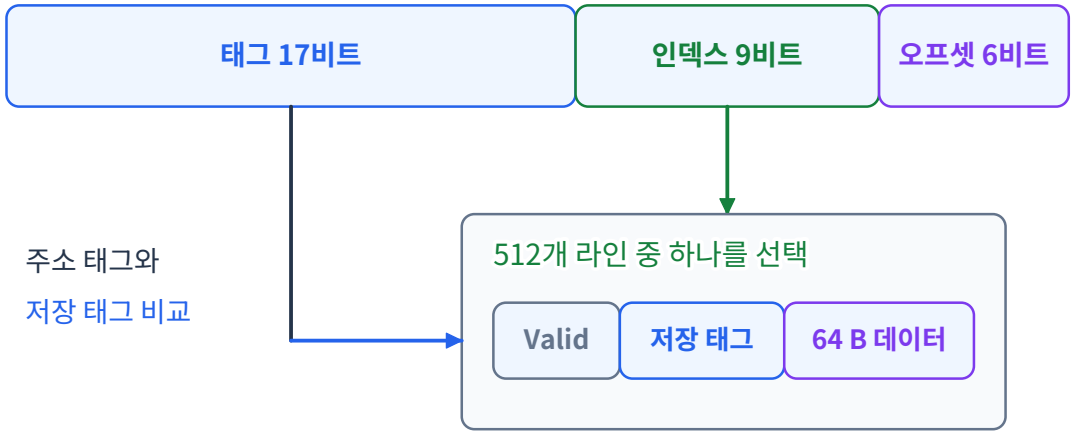
직접 사상은 메모리 블록이 들어갈 캐시 위치가 하나로 정해지고, 완전 연관은 어느 위치에도 들어갈 수 있다. 집합 연관은 지정된 집합 안 여러 위치 중 하나에 들어간다. write-through는 하위 기억장치에도 함께 쓰고, write-back은 바뀐 블록을 표시한 뒤 교체 시 내려보낸다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 캐시 주소 분해와 적중 판정

직접 사상 캐시: 태그·인덱스·블록 내 위치

예: 32비트 주소 · 캐시 데이터 32 KiB · 블록 64 B



라인=메모리 블록 번호 mod 512 · Valid=1이고 태그가 같으면 적중

데이터 용량만 32 KiB로 가정한 직접 사상 예이다. 64 B 오프셋은 6비트, 512라인 인덱스는 9비트이며 나머지 17비트가 태그다.

공식 1

$$h = \frac{N_{\text{hit}}}{N_{\text{access}}}, \qquad r_{\text{miss}} = 1 - h$$

h와 r은 0~1의 비율. 퍼센트를 계산에 넣을 때 100으로 나눈다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

공식 2

$$\mathrm{AMAT} = t_{\mathrm{hit}} + (1-h)t_{\mathrm{penalty}}$$

penalty는 miss 때 추가되는 시간으로 정의했을 때의 식.

개념	핵심 구분	대표 예
시간적 지역성	같은 주소를 곧 재사용	반복문 안 같은 변수
공간적 지역성	인접 주소를 연속 사용	배열을 순서대로 읽기
write-through	하위 저장에도 즉시 반영	일관성 관리가 비교적 단순
write-back	교체 등 때 나중에 반영	dirty 표시 필요

함정 체크 — 여기서 틀린다

- 캐시 용량을 늘리면 언제나 hit 시간이 줄어든다는 보장은 없다.
- miss penalty가 총 miss 시간인지 추가 시간인지 반드시 구별한다.

짧은 예제

hit 1ns, 적중률 90%, 추가 miss penalty 20ns이면 평균 시간은?

1 miss율은 0.1이다.

2 **AMAT=1+0.1×20**이다.

답 → 3ns

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

가리고 답하기

같은 명령어가 반복문에서 계속 사용되는 것은 어떤 지역성인가?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

D19 인터럽트·DMA·입출력 방식

B 선택

DAY 5 · 오후 · 첫 학습 15분 · 공식 0개

폴링은 CPU가 장치 상태를 반복 확인하는 방식이다. 인터럽트는 장치나 사건이 CPU에 처리가 필요함을 알리는 방식이다. 인터럽트가 발생하면 필요한 실행 상태를 보존하고 처리 루틴으로 이동한 뒤 복귀한다.

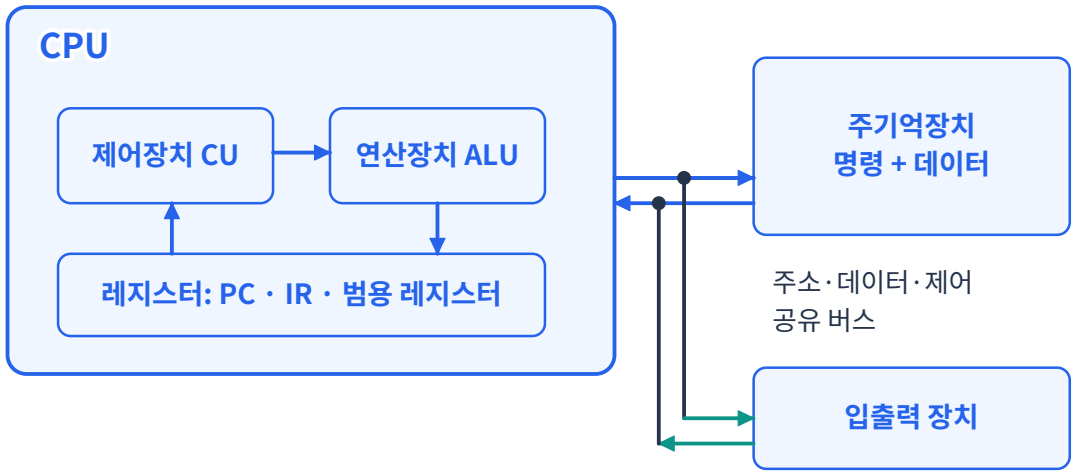
DMA는 대량 데이터 전송의 각 바이트를 CPU가 일일이 처리하지 않도록 별도 제어기가 메모리와 장치 사이 전송을 수행하는 방식이다. CPU는 초기 설정이나 완료 처리에 관여할 수 있다. 따라서 DMA는 CPU가 전혀 관여하지 않는 모든 작업이라는 뜻이 아니다.

인터럽트는 예외·우선순위·마스크 가능 여부 등의 분류가 있지만, 5일 학습에서는 폴링과의 차이 및 DMA의 목적을 먼저 외운다. 전기 계측 데이터 취득을 컴퓨터가 처리한다는 맥락에서 각 역할을 연결한다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

그림 · 컴퓨터의 기본 구성과 명령 실행

폰 노이만 구조: 명령과 데이터를 같은 메모리에서 읽는다



명령 인출 → 해독 → 피연산자 읽기 → 실행 → 결과 저장

PC: 다음 명령 주소 · IR: 현재 명령 · ALU: 산술·논리 연산

이 도식은 공유 메모리의 논리 구조를 단순화한다. 실제 구현에서는 캐시·버스·입출력 제어기가 사이에 놓일 수 있다.

방식	누가 언제 확인하는가	주요 특징
폴링	CPU가 반복 확인	구조 단순, 확인하는 시간 소요
인터럽트	필요한 사건이 CPU에 통지	상태 보존·처리·복귀
DMA	전송 제어기가 대량 전송	CPU의 반복 데이터 이동 부담 감소

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

함정 체크 — 여기서 틀린다

- 인터럽트가 오면 무조건 모든 현재 상태를 잃는 것이 아니다.
- DMA도 초기 주소·길이 설정 등에는 CPU가 관여할 수 있다.

짧은 예제

CPU가 장치 완료 비트를 계속 읽는 방식은?

1 장치가 먼저 통지하는 형태가 아니라 CPU가 반복 확인한다.

답 → 폴링

가리고 답하기

DMA의 주요 목적은?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D20 ADC·DAC·표본화의 최소 개념

B 선택

DAY 5 · 오후 · 첫 학습 15분 · 공식 2개

ADC는 아날로그 입력을 디지털 코드로, DAC는 디지털 코드를 아날로그 출력으로 바꾼다. ADC의 비트 수가 커지면 같은 입력 범위를 더 많은 구간으로 나누어 양자화 간격을 줄일 수 있다. 분해능 증가와 변환 속도 증가는 같은 말이 아니다.

이상적인 n비트 ADC에는 2^n개 코드가 있다. 입력 범위를 균등 구간으로 나눈 ADC의 1LSB는 범위/2^n으로 정의한다. DAC에서 최댓값 코드를 정확히 정격 최대전압에 맞추는 끝점 정의를 쓰면 분모가 2^n-1인 식도 있으므로 장치와 문제 정의를 확인한다.

대역 제한 신호를 복원하려면 표본화 주파수를 신호 최고 주파수의 두 배보다 크게 잡는 기본 조건이 필요하다. 조건을 만족하지 못하면 고주파가 다른 낮은 주파수처럼 보이는 aliasing이 생길 수 있다. 실제 설계에서는 경계와 필터 여유가 필요하다.

공식 1

$$\Delta = \frac{V_{\max} - V_{\min}}{2^n}$$

이상적인 균등 양자화 ADC의 구간 폭. 과부하·포화가 없는 유효 입력 범위의 이상적인 반올림 모델에서는 양자화 오차가 ±Δ/2 이내다.

공식 2

$$f_s > 2f_{\max}$$

대역 제한 신호의 기본 표본화 조건. 등호 경계에서 모든 신호가 항상 안전하다고 외우지 않는다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

함정 체크 — 여기서 틀린다

- ADC와 DAC의 변환 방향을 바꾸지 않는다.
- 2^n과 2^n-1은 정의에 따라 구별해야 한다.

짧은 예제

입력 범위가 0~8V인 이상적인 3비트 균등 ADC의 1LSB는?

1 코드는 8개다.

2 범위 8V를 8구간으로 나눈다.

답 → 1V

가리고 답하기

샘플링 전에 신호의 고주파 성분을 제한하는 필터의 목적은?

정답 보기 → (오른쪽 패드에 먼저 답을 쓰고 누르기)

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3

공식 총정리 · 백지 테스트

다음 날 아침 30분 루틴: 왼쪽을 가리고 오른쪽 패드에 단원별 공식을 전부 써본 뒤 대조. 틀린 공식만 빨간색으로 다시 3번.

D01 A

$$0 \le x \le 2^n - 1$$

n비트 부호 없는 정수의 범위. 가능한 값의 개수는 2^n개다.

D01 A

$$1\ \text{byte} = 8\ \text{bit}$$

KiB는 1024 byte, KB는 원칙적으로 1000 byte. 시험에서 제시한 단위 정의를 우선한다.

D02 B

$$-2^{n-1} \le x \le 2^{n-1} - 1$$

n비트 2의 보수 정수 범위. 예: 8비트는 -128~127.

D03 A

$$G = B \oplus (B \gg 1)$$

B는 부호 없는 이진 비트열, >>1은 오른쪽 1비트 논리 시프트다.

D03 A

$$g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i$$

그레이 변환에서는 오른쪽 식의 두 비트가 모두 원래 이진수 비트다.

D04 A

$$A \oplus B = \overline{A}B + A\overline{B}$$

2입력 XOR. A=B이면 0, A≠B이면 1.

D04 A

$$\overline{A \oplus B} = AB + \overline{A}\,\overline{B}$$

2입력 XNOR. 입력이 같은지 판단하는 기본 논리다.

D05 A

$$\overline{AB} = \overline{A} + \overline{B}$$

곱의 반전은 각 항을 반전한 논리합.

D05 A

$$\overline{A + B} = \overline{A}\,\overline{B}$$

합의 반전은 각 항을 반전한 논리곱.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D05 A

$$\overline{A}\overline{A} = \overline{A}, \qquad \overline{A + A} = \overline{A}$$

NAND 또는 NOR의 두 입력을 묶으면 인버터가 된다.

D06 A

$$A + AB = A, \qquad A(A + B) = A$$

흡수법칙.

D06 A

$$A + \overline{A}B = A + B$$

A=0일 때 B, A=1일 때 1인 함수다.

D06 A

$$AB + A\overline{B} = A$$

B+NOT B=1로 묶는다.

D07 A

$$F(A, B, C) = \sum m(0, 1, 4, 5) = \overline{B}$$

ABC 순서에서 B=0인 네 칸을 묶은 예. A와 C는 변하므로 사라진다.

D08 A

$$S = A \oplus B, \qquad C = AB$$

반가산기. C는 출력 자리올림.

D08 A

$$S = A \oplus B \oplus C_{\text{in}}$$

전가산기의 합.

D08 A

$$C_{\text{out}} = AB + C_{\text{in}}(A \oplus B)$$

전가산기 캐리. AB+ACin+BCin과 동치다.

D09 A

$$D = A \oplus B, \qquad B_{\text{out}} = \overline{A}B$$

반감산기. A가 피감수, B가 감수다.

D09 A

$$D = A \oplus B \oplus B_{\text{in}}$$

전감산기의 차.

D09 A

$$B_{\text{out}} = \overline{A}B + \overline{A}B_{\text{in}} + BB_{\text{in}}$$

전감산기의 빌림. 입력 순서 A-B-Bin을 유지한다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D10 B

$$Y = \overline{S}I_0 + SI_1$$

2:1 MUX. S=0이면 I0, S=1이면 I1.

D10 B

$$N = 2^k$$

입력이 2^k개인 MUX는 일반적으로 k개의 선택 비트가 필요하다.

D11 A

$$Q_{n+1} = D$$

D 플립플롭의 특성식. 지정된 활성 에지에서 갱신한다.

D11 A

$$Q_{n+1} = T \oplus Q_n$$

T 플립플롭. T=1이면 반전.

D11 A

$$Q_{n+1} = J\overline{Q_n} + \overline{K}Q_n$$

JK 플립플롭의 특성식.

D12 B

$$N_{\max} = 2^n, \qquad f_{\text{out}} = \frac{f_{\text{clk}}}{2^n}$$

2진 분주 카운터의 최상위 단계 기준. MOD-N 전체 반복 주기는 N개의 클럭이다.

D18 A

$$h = \frac{N_{\text{hit}}}{N_{\text{access}}}, \qquad r_{\text{miss}} = 1 - h$$

h와 r은 0~1의 비율. 퍼센트를 계산에 넣을 때 100으로 나눈다.

D18 A

$$\text{AMAT} = t_{\text{hit}} + (1-h)t_{\text{penalty}}$$

penalty는 miss 때 추가되는 시간으로 정의했을 때의 식.

D20 B

$$\Delta = \frac{V_{\max} - V_{\min}}{2^n}$$

이상적인 균등 양자화 ADC의 구간 폭. 과부하·포화가 없는 유효 입력 범위의 이상적인 반올림 모델에서는 양자화 오차가 ±Δ/2 이내다.

D20 B

$$f_s > 2f_{\max}$$

대역 제한 신호의 기본 표본화 조건. 등호 경계에서 모든 신호가 항상 안전하다고 외우지 않는다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

함정 총정리 · 시험 당일 아침용

10.24 아침에는 새 내용을 보지 말고 이 목록만 훑습니다. 읽자마자 이유가 안 떠오르는 줄에만 표시.

D01	2^n은 최대 숫자가 아니라 패턴의 개수다.
D01	BCD 4비트의 1010~1111은 정상적인 한 자리 십진 숫자 표현으로 쓰지 않는다.
D02	비트 수가 달라지면 같은 모양의 비트열도 해석이 달라질 수 있다.
D02	최상위 캐리와 부호 있는 오버플로는 같은 개념이 아니다.
D03	그레이코드는 오류를 자동으로 정정하는 코드라는 뜻이 아니다.
D03	변환 방향에 따라 XOR에 사용하는 상위 비트가 다르다.
D04	OR와 XOR는 입력 11에서만 출력이 다르다.
D04	NAND는 AND 결과 전체의 반전이다. 입력 하나만 반전하는 것이 아니다.
D05	AND 하나 또는 OR 하나만 반복해서는 NOT을 만들 수 없으므로 범용이 아니다.
D05	출력 전체를 반전하는 괄호의 범위를 끝까지 확인한다.
D06	불 대수에서 **A+A=2A**로 쓰지 않는다.
D06	최소항 번호는 행·열의 위치가 아니라 변수 비트열을 십진수로 읽은 값이다.
D07	3칸·6칸 묶음이나 L자 모양 묶음은 허용되지 않는다.
D07	일부 해설의 위치표의 **AB=11,CD=10**은 m16이 아니라 m14다. 4변수 번호는 0~15다.
D08	가산기 합은 OR가 아니라 XOR다.
D08	입력 캐리와 출력 캐리는 다른 신호다.
D09	A와 B를 바꾸면 차 비트는 같을 수 있지만 빌림은 달라진다.
D09	출력 빌림은 위 자리에서 2를 빌려왔다는 표시이며 음수 부호 비트와 다르다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D10	데이터 입력 개수와 선택 입력 개수를 혼동하지 않는다. ○
D10	일반 인코더는 여러 활성 입력이 동시에 주어지면 모호할 수 있다. ○
D11	래치와 에지 트리거 플립플롭의 반응 시간을 구별한다. ○
D11	순서회로에는 초기 상태와 클럭 조건이 필요하다. ○
D12	MOD-10은 10비트가 아니라 최소 4비트가 필요하다. ○
D12	초기값이 000이 아닐 수 있다. 리셋 조건과 활성 에지를 확인한다. ○
D13	NMOS와 PMOS의 ON 조건을 같은 것으로 외우지 않는다. ○
D13	논리게이트 회로와 아날로그 MOSFET 증폭기 동작 영역을 섞지 않는다. ○
D14	폰 노이만 구조를 오직 한 명령만 처리할 수 있는 모든 현대 구현이라는 뜻으로 확대하지 않는다. ○
D14	CPU와 컴퓨터 전체를 같은 구성요소로 보지 않는다. ○
D15	operand와 opcode를 뒤집어 외우지 않는다. ○
D15	주소 100과 그 주소에 저장된 값 M[100]은 다르다. ○
D16	PC가 다음 명령의 내용 자체를 저장한다고 하지 않는다. ○
D16	명령어 인출과 피연산자 인출은 같은 종류의 정보가 아니다. ○
D17	정적 RAM의 정적은 비휘발성이라는 뜻이 아니다. ○
D17	캐시와 가상기억장치는 다른 개념이다. ○
D18	캐시 용량을 늘리면 언제나 hit 시간이 줄어든다는 보장은 없다. ○
D18	miss penalty가 총 miss 시간인지 추가 시간인지 반드시 구별한다. ○
D19	인터럽트가 오면 무조건 모든 현재 상태를 잃는 것이 아니다. ○
D19	DMA도 초기 주소·길이 설정 등에는 CPU가 관여할 수 있다. ○
D20	ADC와 DAC의 변환 방향을 바꾸지 않는다. ○

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D20	2^n과 2^n-1은 정의에 따라 구별해야 한다.	◯

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

백지 회상 문항

답은 오른쪽 패드에. 계산보다 공식·조건·단위를 꺼내는 연습입니다. 번호를 누르면 정답으로 이동.

D-P01 부호 없는 5비트의 최댓값은?
D01

D-P02 2의 보수에서 양수 둘의 합이 음수로 해석되면?
D02

D-P03 그레이코드에서 인접 상태가 바뀔 때 달라지는 비트 수는?
D03

D-P04 입력 11에서 OR와 XOR 출력은?
D04

D-P05 NAND만으로 NOT을 만드는 연결법은?
D05

D-P06 A+AB의 간소화 결과는?
D06

D-P07 카르노맵에서 1을 6개 직사각형으로 묶어도 되는가?
D07

D-P08 반가산기에 **A=B=1**일 때 합과 캐리는?
D08

D-P09 반감산기의 빌림 식을 쓰시오.
D09

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D-P10 전감산기 **A=1,B=1,Bin=1**의 차와 빌림은?
D09

D-P11 16:1 MUX의 선택 비트 수는?
D10

D-P12 JK 플립플롭의 입력이 11이면?
D11

D-P13 MOD-10 카운터에 필요한 최소 플립플롭 수는?
D12

D-P14 CMOS NAND의 NMOS 연결은 직렬인가 병렬인가?
D13

D-P15 폰 노이만 구조의 핵심 저장 원리는?
D14

D-P16 opcode와 operand가 각각 나타내는 것은?
D15

D-P17 다음 명령의 주소와 현재 명령 자체를 담는 레지스터는?
D16

D-P18 SRAM은 refresh가 필요한가? 비휘발성인가?
D17

D-P19 배열의 인접 원소를 차례로 읽는 성질은?
D18

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D-P20 write-back 캐시가 수정 여부를 표시하는 비트는?
D18

D-P21 CPU가 상태를 반복 확인하는 입출력 방식은?
D19

D-P22 ADC 비트 수가 1 증가하면 코드 수는 어떻게 되는가?
D20

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

정답 모음

ID를 누르면 본문으로 돌아갑니다.

단원별 가리고 답하기

D01 1010(2)는 십진수와 16진수로 각각 무엇인가?

십진수 10, 16진수 A다. 왼쪽 두 1의 자릿값 8과 2를 더한다.

D02 8비트 2의 보수의 최소값과 최대값은?

-128과 +127이다.

D03 카르노맵 열을 00,01,10,11 순으로 쓰면 안 되는 이유는?

01과 10이 이웃하면 두 비트가 동시에 바뀌어 카르노맵의 인접성 조건을 어기기 때문이다.

D04 2입력 XOR가 1인 두 입력 조합은?

01과 10이다.

D05 범용 게이트 두 종류는? 입력 두 개를 묶으면 무엇이 되는가?

NAND와 NOR. 두 입력을 같은 신호에 묶으면 NOT 게이트가 된다.

D06 ABC의 순서에서 최소항 m5를 쓰면?

5=101이므로 A·NOT B·C다.

D07 카르노맵의 네 모서리 칸을 하나의 묶음으로 볼 수 있는가?

4변수 맵에서 네 칸이 모두 1 또는 허용되는 무관항이면 가능하다. 양 끝끼리 연결되어 인접하기 때문이다.

D08 반가산기와 전가산기의 입력 개수 차이는?

반가산기는 2개, 전가산기는 입력 캐리를 포함해 3개다.

D09 반감산기에서 A=1,B=0이면 빌림이 필요한가?

아니다. 차는 1, 빌림은 0이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D10 한 입력을 선택된 한 출력 경로로 보내는 것은?

DEMUX다.

D11 JK=00은 어떤 동작인가?

현재 상태 유지다. Q가 0이면 0, 1이면 1을 유지한다.

D12 부호 없는 00110000을 논리 오른쪽으로 세 번 옮기면?

00000110이다. 십진수 48을 8로 나눈 6과 같다.

D13 CMOS NAND에서 A=B=1이면 출력이 내려가는 이유는?

직렬 NMOS 두 개가 모두 켜져 출력에서 접지까지 전류 경로가 만들어지기 때문이다.

D14 명령어를 해독하고 각 장치에 동작 신호를 보내는 CPU 구성요소는?

제어장치다.

D15 operand가 실제 값일 수도 있는 대표 주소지정 방식은?

즉시 주소지정이다.

D16 기억장치와 주고받는 데이터가 잠시 저장되는 레지스터의 두 흔한 이름은?

MDR 또는 MBR다.

D17 DRAM이 refresh를 필요로 하는 이유는?

저장된 전하가 누설되어 정보를 잃을 수 있어 주기적으로 복원해야 하기 때문이다.

D18 같은 명령어가 반복문에서 계속 사용되는 것은 어떤 지역성인가?

시간적 지역성이다.

D19 DMA의 주요 목적은?

대량 전송에서 CPU의 데이터 이동 부담을 줄이는 것이다.

D20 샘플링 전에 신호의 고주파 성분을 제한하는 필터의 목적은?

aliasing을 줄이기 위해서다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

백지 회상 문항 정답

D-P01 31
2^5개의 값에 0이 포함된다.

D-P02 오버플로
같은 부호의 덧셈 결과가 반대 부호다.

D-P03 1비트
반사형 그레이코드의 핵심 성질이다.

D-P04 **OR=1, XOR=0**
OR는 하나 이상 1, XOR는 두 입력이 다른 경우 1이다.

D-P05 두 입력을 같은 신호에 연결
NOT(AA)=NOT A다.

D-P06 A
흡수법칙이다.

D-P07 안 된다
묶음 크기는 1,2,4,8 등 2의 거듭제곱이어야 한다.

D-P08 합0, 캐리1
산술 합 2는 이진수10이다.

D-P09 NOT A·B
A-B에서 **A=0**,**B=1**일 때만 빌림이 필요하다.

D-P10 차1, 빌림1
1-1-1=-1이다. 2를 빌려오면 이 자리 차는1이다.

D-P11 4
2^4=16이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

D-P12 반전
현재 Q의 보수가 다음 Q다.

D-P13 4개
2^3<10≤2^4이다.

D-P14 직렬
모든 입력이 1일 때만 직렬 NMOS가 모두 켜져 접지로 통하는 풀다운 경로가 형성된다.

D-P15 명령어와 데이터를 기억장치에 저장하는 프로그램 내장 방식
기본 모델은 기억공간·경로를 공유한다.

D-P16 동작 종류 / 연산 대상 또는 그 지정 정보
operand가 항상 주소나 항상 데이터로 고정되지는 않는다.

D-P17 PC / IR
주소와 내용을 구별한다.

D-P18 불필요 / 일반적으로 휘발성
정적이라는 말은 전원 없이도 저장된다는 뜻이 아니다.

D-P19 공간적 지역성
가까운 주소를 연속 사용한다.

D-P20 dirty bit
하위 메모리와 내용이 달라졌는지를 나타낸다.

D-P21 폴링
인터럽트는 사건 발생을 통지하는 방식이다.

D-P22 2배
동일 입력 범위에서는 이상적인 양자화 간격이 절반이다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 ① ② ③

마무리

비공식 학습자료입니다. 시험 출제 범위·비중을 예측하거나 보장하지 않으며, 오류가 있을 수 있습니다. 공식 수험 안내와 표준 교재를 우선하세요.

반가산기·전가산기·반감산기·전감산기는 합/차와 캐리/빌림을 분리해 쓴다.

단순 암기 뒤에는 입력 0과1의 한 조합을 대입하여 스스로 검산한다.

컴퓨터구조는 PC/IR, 주소/값, SRAM/DRAM, 시간/공간 지역성처럼 혼동하기 쉬운 짝을 기준으로 외운다.

WRITE PAD 가리고 → 쓰고 → 확인 → 지우고 다시 1 2 3